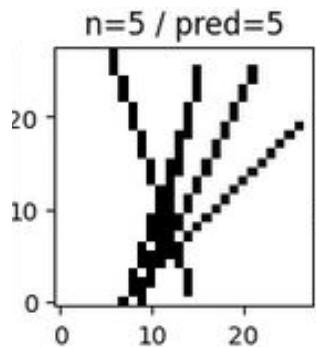
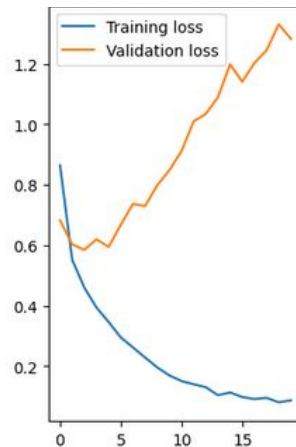
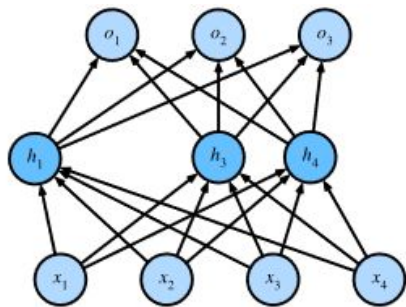


Counting lines...

Mathematics for Big Data

Deep Learning

Mathias LOMMEL





Context

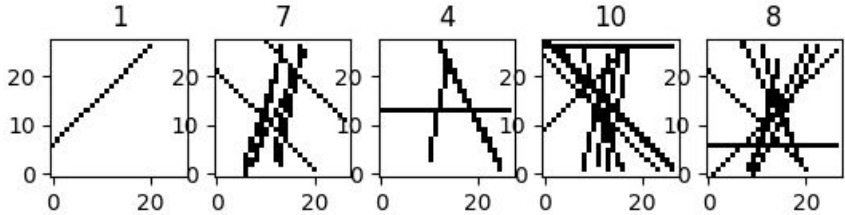


Context

Objective :

→ *Build a dataset*

- *Images*
- *Size 28 x 28*
- *Lines drawn randomly (1 - 10)*



→ *Train a neural network to recognize the number of lines*

Remark :

We have 10 classes (1 to 10 lines) → **Multi-Class classification**

The dataset

Building the dataset

Select an **origin**

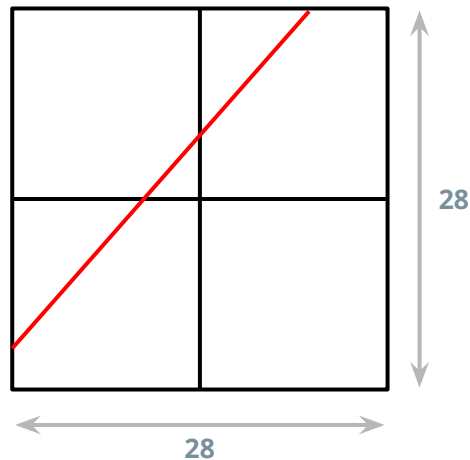
Choose a **slope** randomly : "a"

Choose an **intercept** randomly : "b"

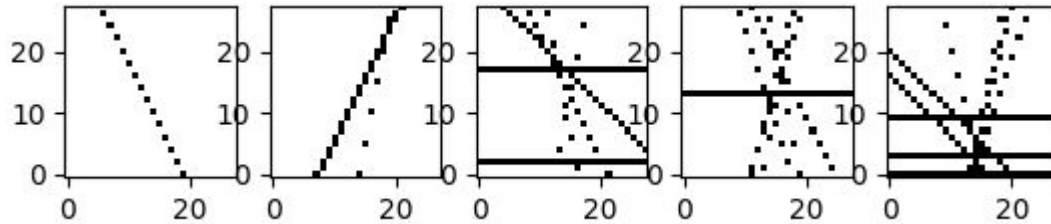
$$\rightarrow \mathbf{y = a.x + b}$$

Add the line to the image

But... The image is represented by a matrix...

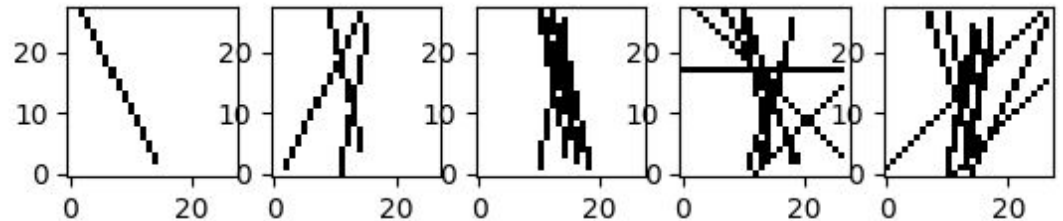


Building the dataset



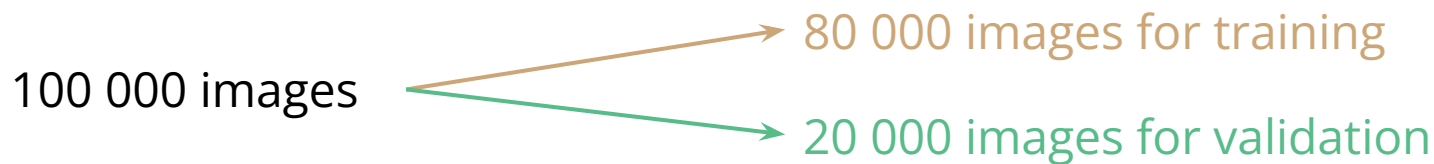
Sometimes the line is made with distant points...

→ We can add points between them !



Building the dataset

Parameters of the dataset :



Uniform dataset : 10 000 images per group

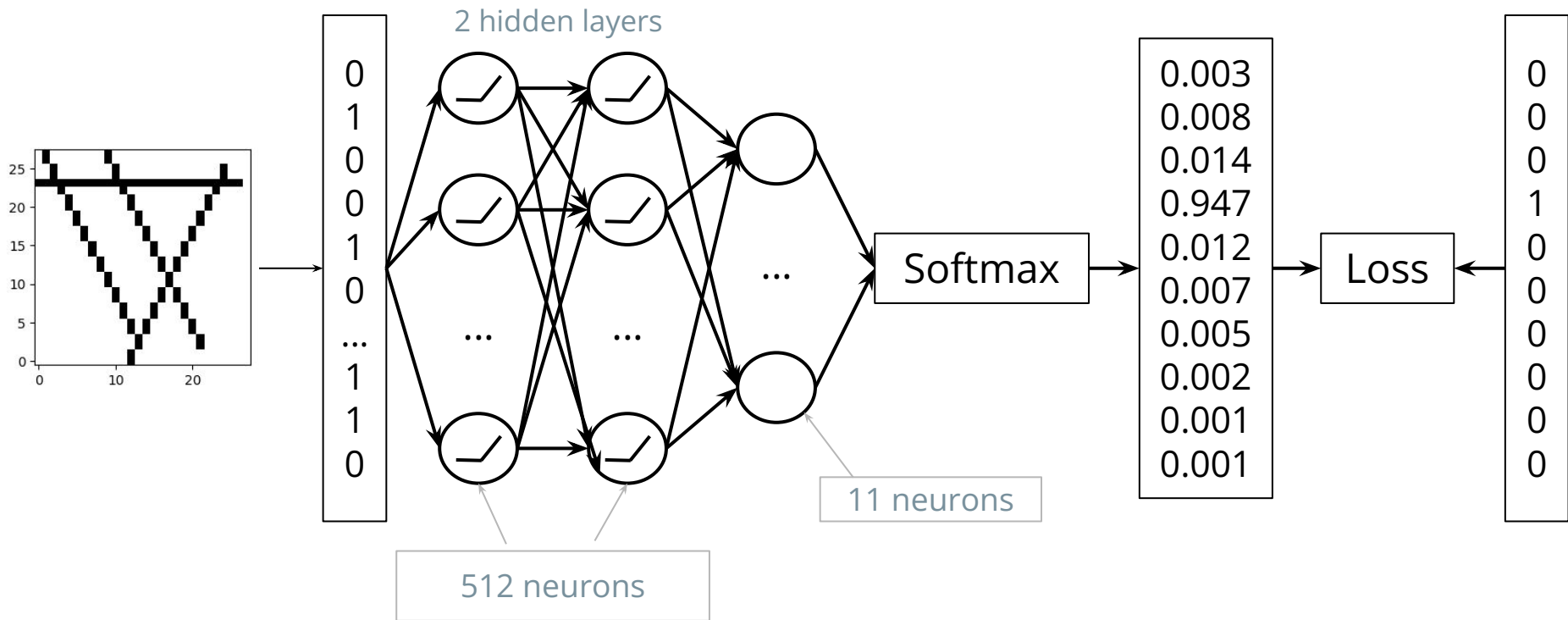
→ Same number of image for each number of lines



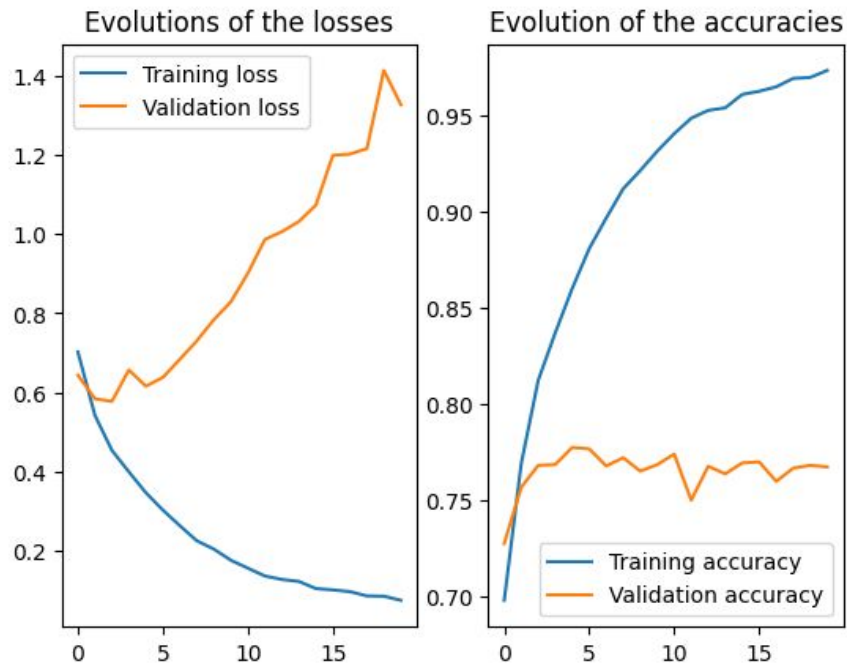
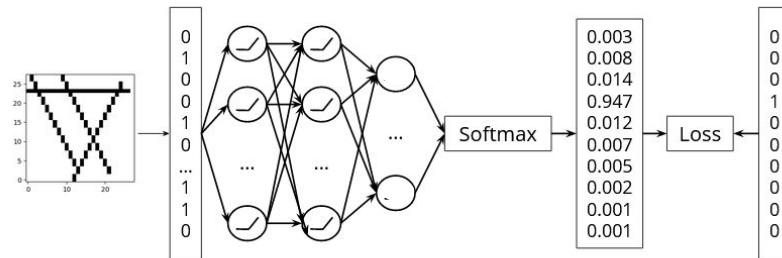
First model



Simple MLP (Multilayer perceptron)



Simple MLP (Multilayer perceptron)



Validation accuracy : 0.768

Time of training : 264.8s



Improvements

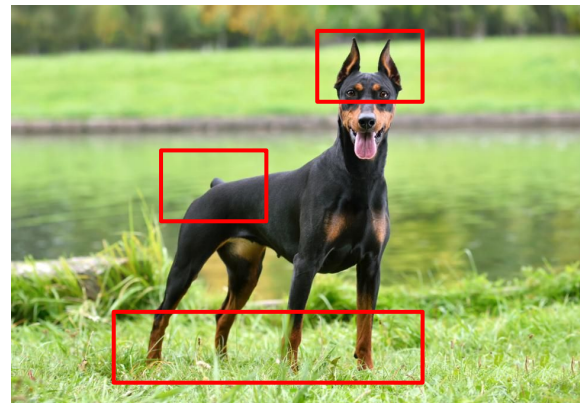
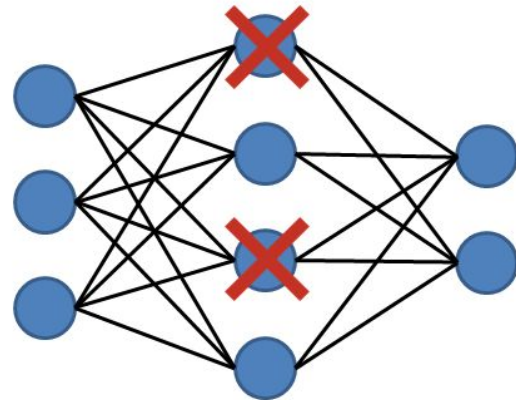
Improvements...

To limit the overfitting, we can apply a so-called
“**Dropout**”

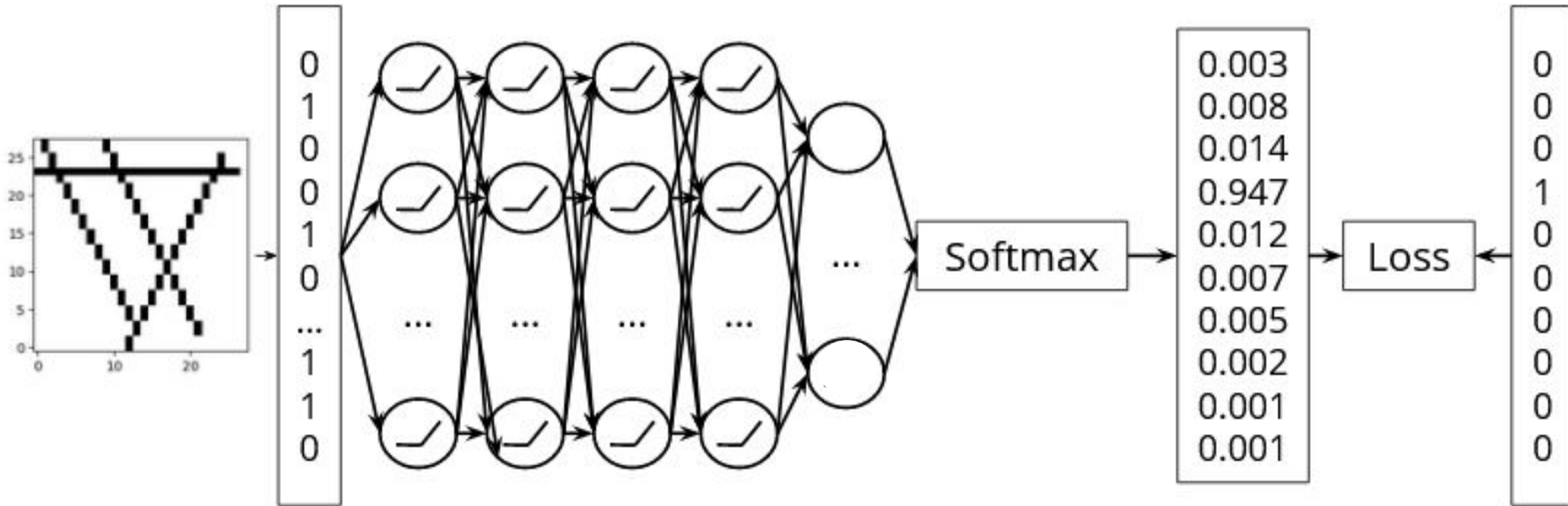
For each forward pass, some neurons will be shut down

The model will have a more general learning

- Some unused neurons will now be used
- New features can be learnt
- The output will depend on new parts of the image

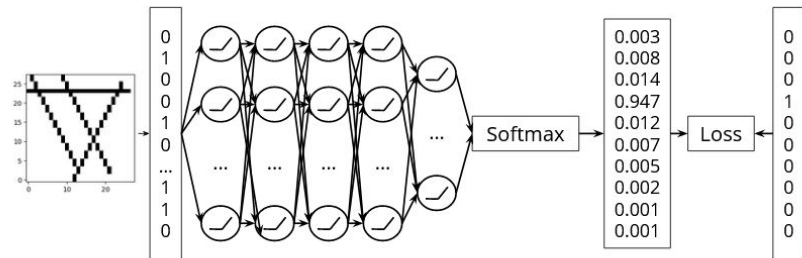
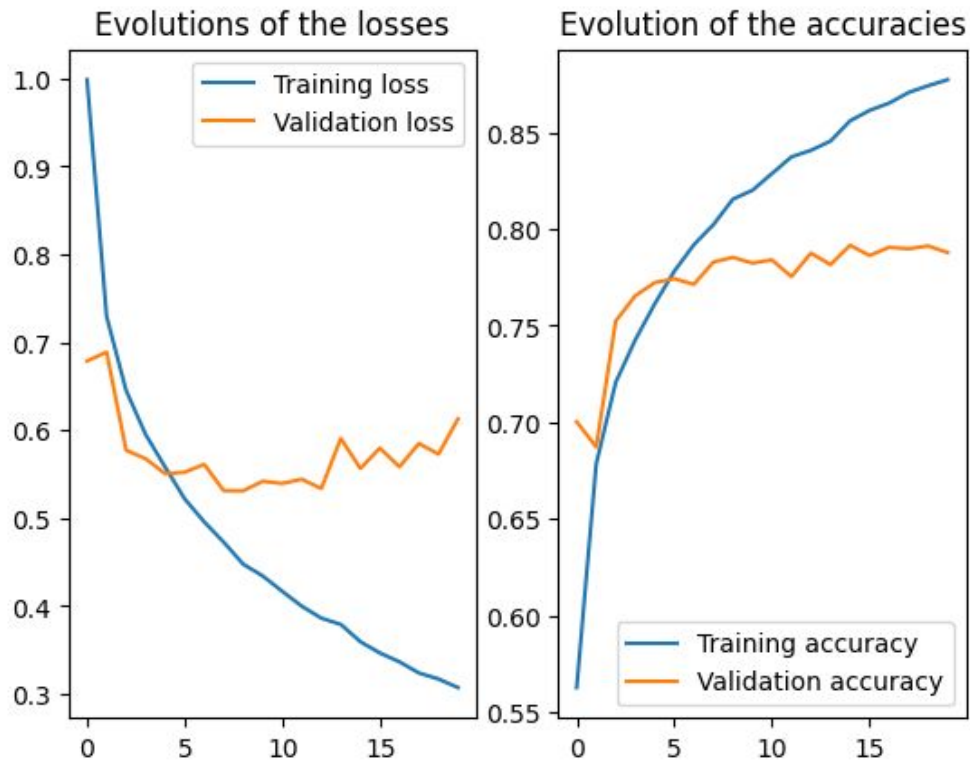


Improvements...



Drop rate : 0.2

Improvements...



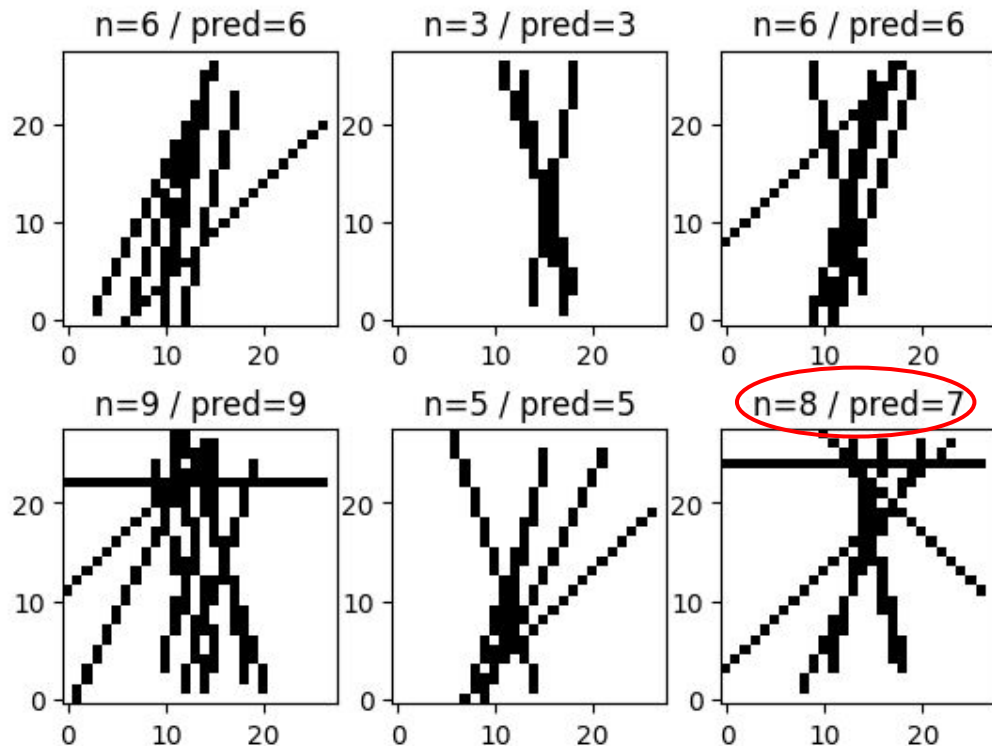
Best accuracy : 0.785

0.017 better !

Training time : 442.7s

But 177.9s slower

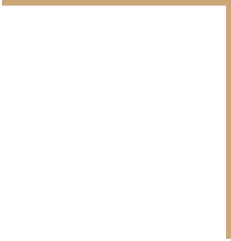
Improvements...



Most of the predictions are good...

... but sometimes...

BUT, the result is still close to the exact label



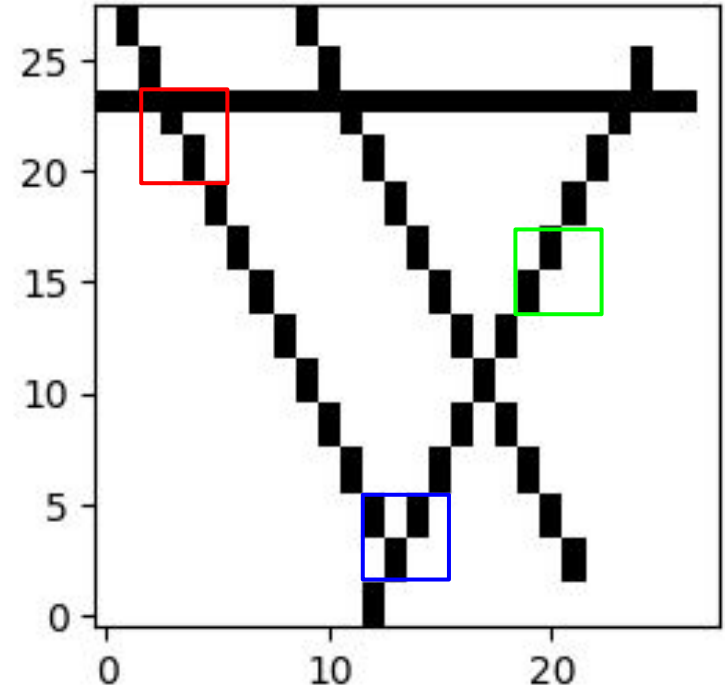
Second model



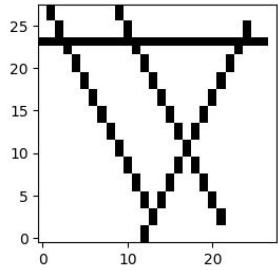
Convolutional Network

This time, we will work directly on the images that we created.

→ This will allow us to consider the spatial shape of the image



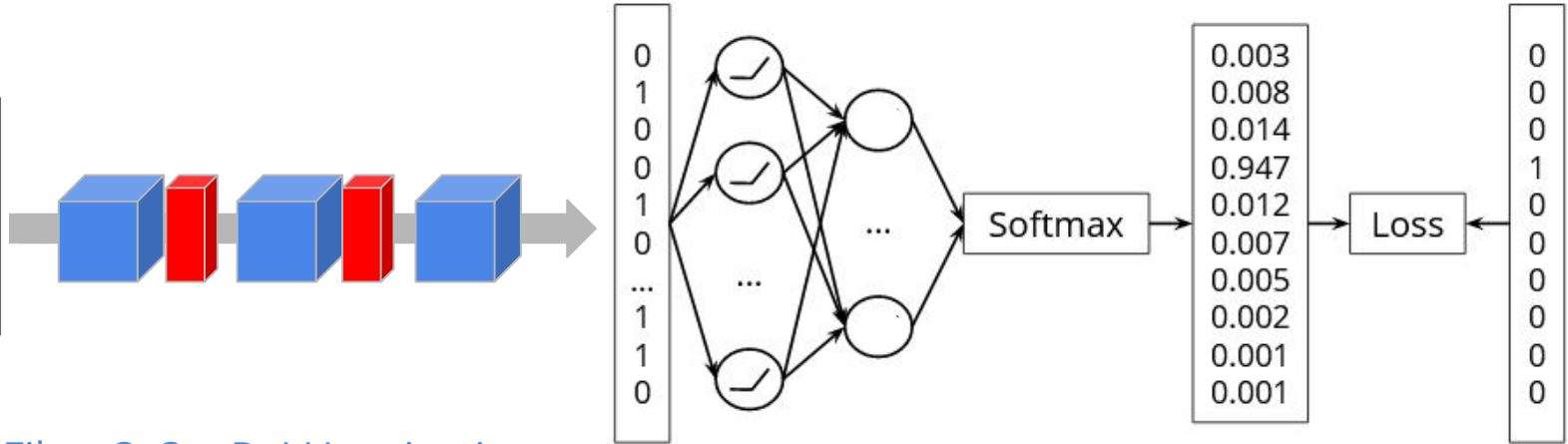
Convolutional Network



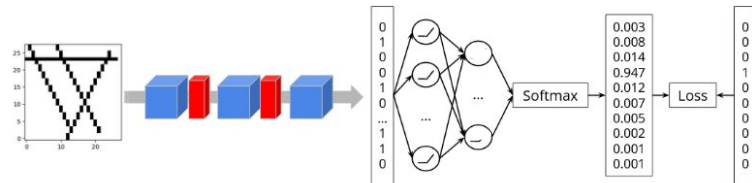
Filter 3x3 + ReLU activation



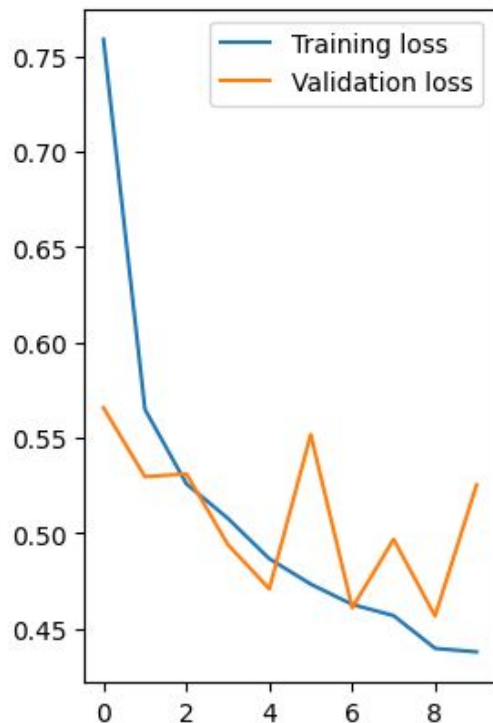
Max Pooling 2x2



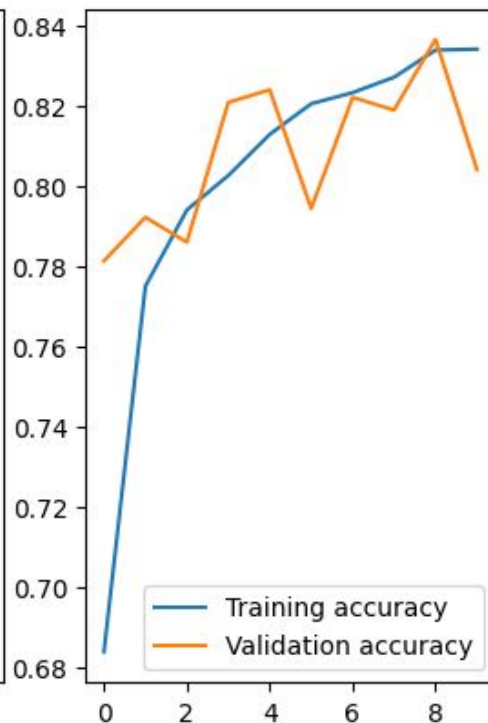
Convolutional Network



Evolutions of the losses



Evolution of the accuracies



Best accuracy : 0.836

0.051 better !

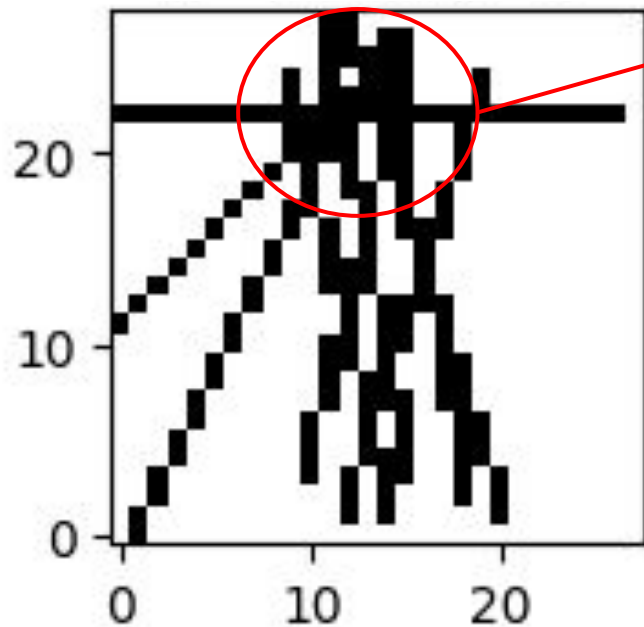
Training time : 715s

But 272.3s slower

+ No overfitting

Improvements

Convolutional



Some regions are difficult to interpret

→ bigger filters may improve our results...

Problem :

bigger filters → more parameters

→ Slower training...

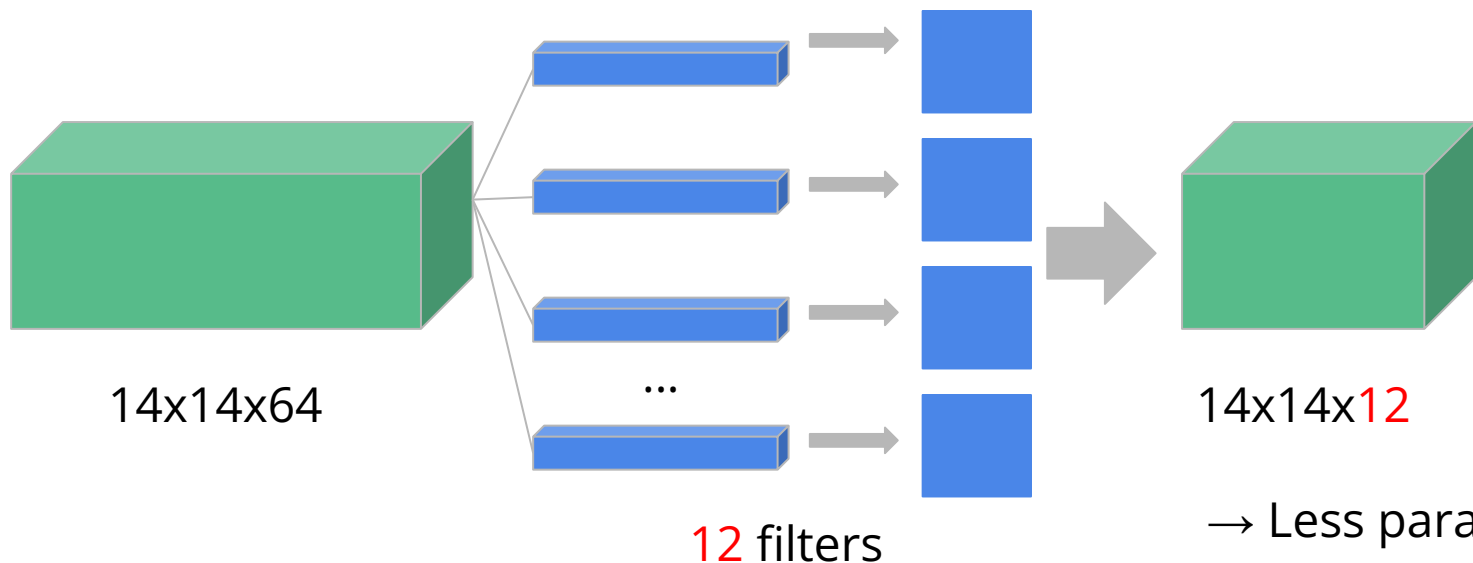
Improvements...

Idea :

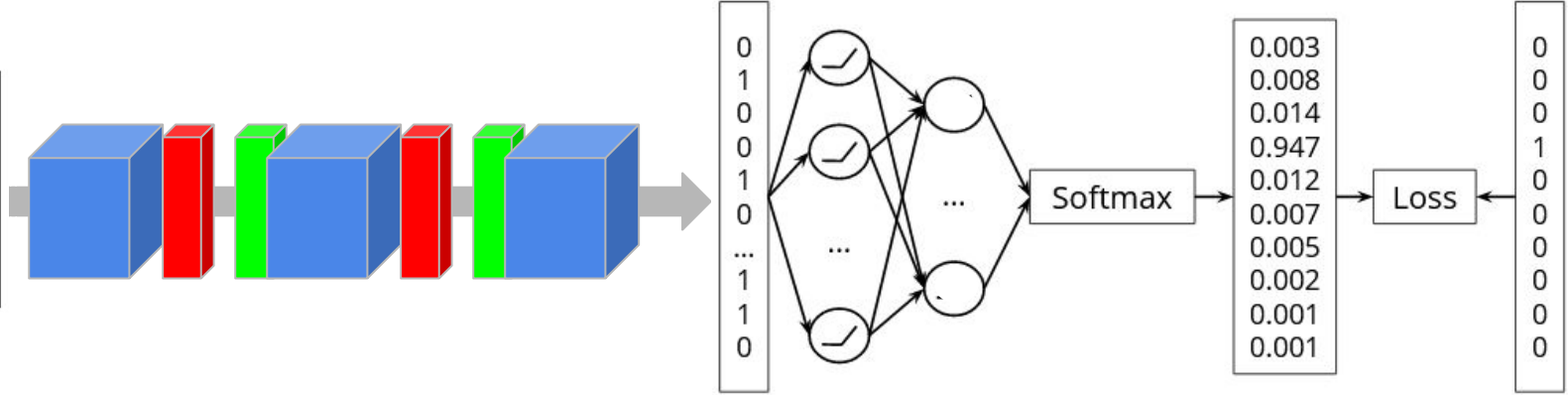
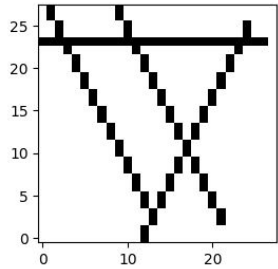
combine 1x1 convolution with 5x5 convolution

→ the 1x1 convolution decreases the number of channels

→ AND we add non linearity !



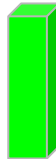
Improvements...



Filter 5x5 + ReLU activation

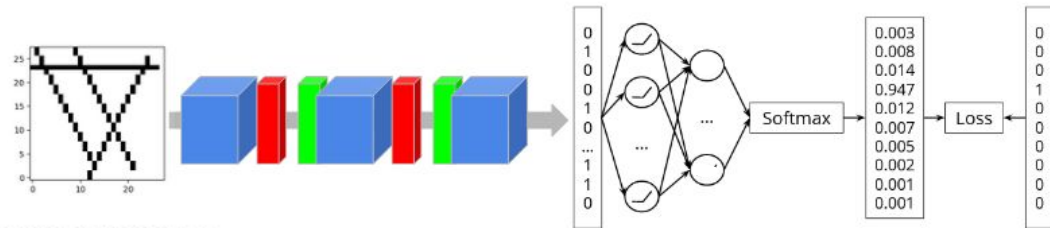


Max Pooling 2x2

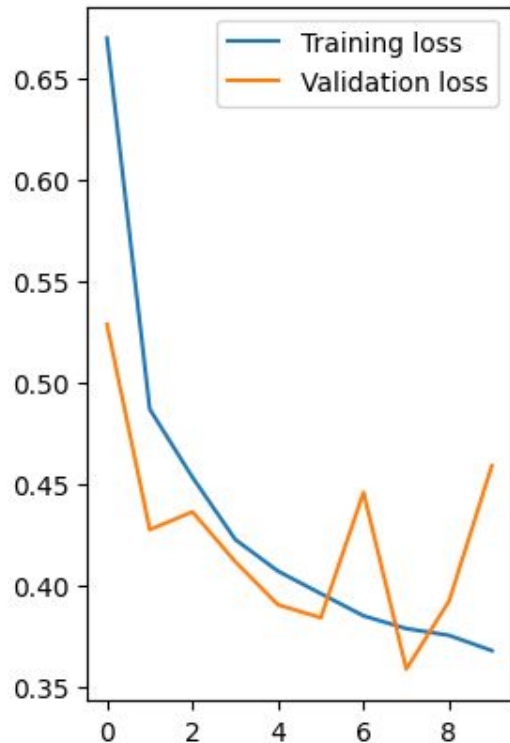


Filter 1x1 + ReLU activation

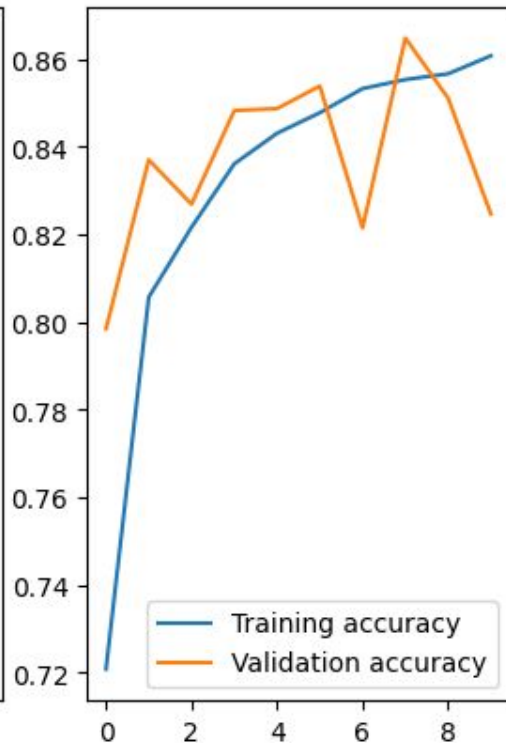
Improvements...



Evolutions of the losses



Evolution of the accuracies



Best accuracy : 0.865

0.029 better

Training time : 939s

But 224s slower

61 612 less parameters

Comparison of the models

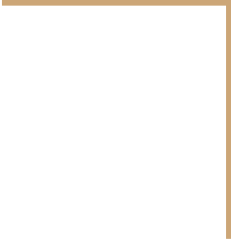
Comparison of the models

	Simple MLP	MLP + Dropout	Convolutional 3x3	Convolutional 5x5
Accuracy	0.768	0.785	0.836	0.865
Minimum loss obtained	0.577	0.531	0.457	0.359
Training time	264.8s	442.7s	715s	939s
Overfitting ?	Yes	No	No	No
Number of parameters	670219 (2.56 MB)	1195531 (4.56 MB)	131019 (511.79 KB)	69407 (271.12 KB)

Conclusion

During this project :

- We have **built a database**
- We learnt how to **put the input into the correct shape**
- We **built** and **trained a model**
- We tried to **improve** a trained model
- We've seen the **performance of Convolutional nets**
- We have **compared** the different models implemented



Thanks for listening

